



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	ELAZEGUI, LORENZO GABRIEL P.
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements

- Loops (for loop)
 - Functions / methods
2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... x + v
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

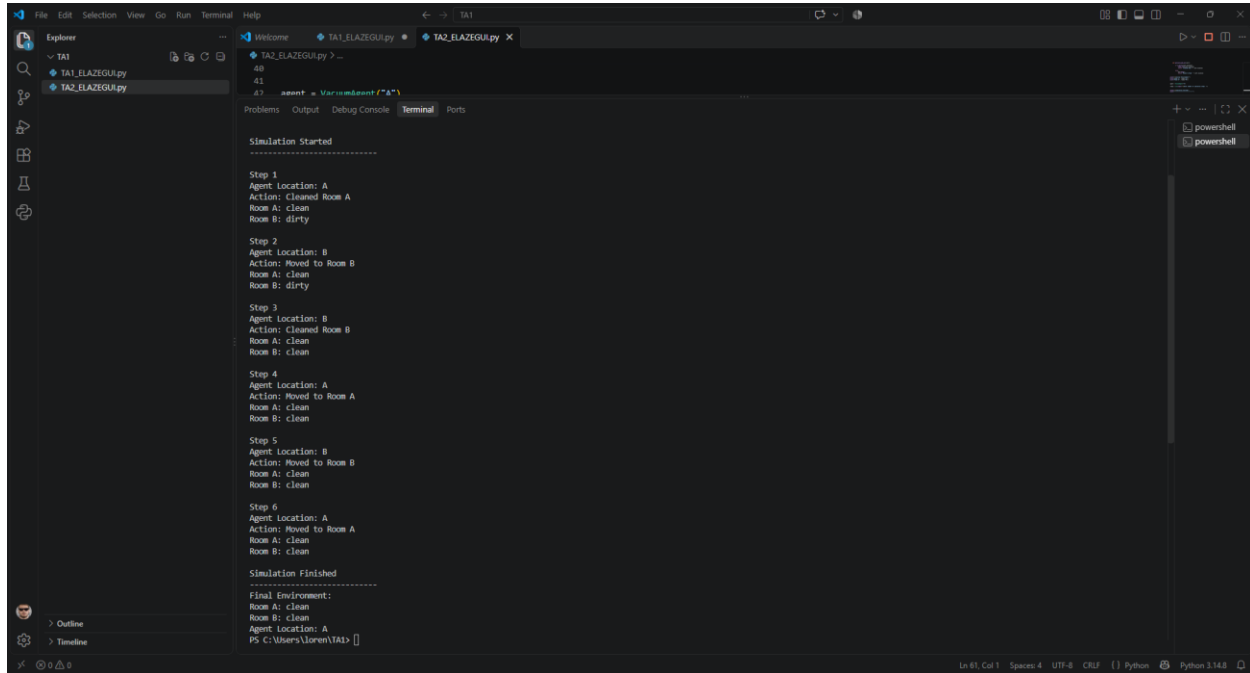
Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
- The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment



The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of a Python script named 'TA1_ELAZEGUI.py'. The script simulates an agent cleaning two rooms, A and B, over six steps. The agent starts in room A, cleans it, moves to room B, cleans it, moves back to room A, cleans it, moves to room B, cleans it, and finally moves back to room A. The simulation ends with a 'Final Environment' summary showing both rooms clean and the agent in room A.

```
TA1_ELAZEGUI.py
40
41
42 agent = MarrowAgent("A")
Problems Output Debug Console Terminal Ports

Simulation Started
-----
Step 1
Agent Location: A
Action: Cleaned Room A
Room A: clean
Room B: dirty

Step 2
Agent Location: B
Action: Moved to Room B
Room A: clean
Room B: dirty

Step 3
Agent Location: B
Action: Cleaned Room B
Room A: clean
Room B: clean

Step 4
Agent Location: A
Action: Moved to Room A
Room A: clean
Room B: clean

Step 5
Agent Location: B
Action: Moved to Room B
Room A: clean
Room B: clean

Step 6
Agent Location: A
Action: Moved to Room A
Room A: clean
Room B: clean

Simulation Finished
-----
Final Environment:
Room A: clean
Room B: clean
Agent Location: A
PS C:\Users\loren\TA1>
```

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
File Edit Selection View Go Run Terminal Help
TA1 TA2_ELAZEGUI.py
TA1 TA2_ELAZEGUI.py
75
76 for ctan in range(1, ctanc + 1):
Enter the state of Room A (dirty/clean): clean
Enter the state of Room B (dirty/clean): dirty
Enter the state of Room C (dirty/clean): clean

Initial Environment:
Room A: clean
Room B: dirty
Room C: clean

Enter number of simulation steps: 6

Simulation Started

Text-Based Grid:
+-----+
| [A*] CLEAN |
| [B] DIRTY  |
| [C] CLEAN  |
+-----+

Step 1
Agent Location: B
Action: Moved to Room B
Room A: clean
Room B: dirty
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] DIRTY |
| [C] CLEAN  |
+-----+

Step 2
Agent Location: B
Action: Cleaned Room B
Room A: clean
Room B: clean
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] CLEAN |
| [C] CLEAN  |
+-----+

Step 3
Agent Location: B
Action: All rooms are clean
Room A: clean
Room B: clean
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] CLEAN |
| [C] CLEAN  |
+-----+

Step 4
Agent Location: B
Action: All rooms are clean
Room A: clean
Room B: clean
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] CLEAN |
| [C] CLEAN  |
+-----+

Step 5
Agent Location: B
Action: All rooms are clean
Room A: clean
Room B: clean
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] CLEAN |
| [C] CLEAN  |
+-----+

Step 6
Agent Location: B
Action: All rooms are clean
Room A: clean
Room B: clean
Room C: clean

Text-Based Grid:
+-----+
| [A] CLEAN  |
| [B*] CLEAN |
| [C] CLEAN  |
+-----+

Simulation Finished

Final Environment:
Room A: clean
Room B: clean
Room C: clean
Agent Location: B
PS C:\Users\Loren\TA1>
```

```

import random

rooms = {
    "A": input("Enter the state of Room A (dirty/clean): ").lower(),
    "B": input("Enter the state of Room B (dirty/clean): ").lower(),
    "C": input("Enter the state of Room C (dirty/clean): ").lower()
}

def is_dirty(room):
    return rooms[room] == "dirty"

def clean_room(room):
    rooms[room] = "clean"

def display_grid(agent_location):
    print("\nText-Based Grid:")
    print("+-----+")

    for room in rooms:
        state = rooms[room].upper()

        if room == agent_location:
            print(f"| [{room}*] {state:<6}|")
        else:
            print(f"| [{room}] {state:<6}|")

    print("+-----+")

class VacuumAgent:

    def __init__(self, location):
        self.location = location

    def move_to_random_dirty_room(self):
        dirty_rooms = [
            room for room in rooms
            if is_dirty(room)
        ]

        if dirty_rooms:
            self.location = random.choice(dirty_rooms)
            return True

        return False

```

```

def perceive_and_act(self):

    if is_dirty(self.location):
        clean_room(self.location)
        return "Cleaned Room " + self.location

    else:
        if self.move_to_random_dirty_room():
            return "Moved to Room " + self.location
        else:
            return "All rooms are clean"

print("\nInitial Environment:")
for room in rooms:
    print("Room", room + ":", rooms[room])

agent = VacuumAgent("A")

steps = int(input("\nEnter number of simulation steps: "))

print("\nSimulation Started")
print("-----")

display_grid(agent.location)

for step in range(1, steps + 1):

    action = agent.perceive_and_act()

    print("\nStep", step)
    print("Agent Location:", agent.location)
    print("Action:", action)

    for room in rooms:
        print("Room", room + ":", rooms[room])

    display_grid(agent.location)

print("\nSimulation Finished")
print("-----")

print("Final Environment:")

```

```

for room in rooms:
    print("Room", room + ":", rooms[room])

print("Agent Location:", agent.location)

```

```

C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |

```

Your Code - Laboratory Tasks

```

rooms = {
    "A": input("Enter the state of Room A (dirty/clean): ").lower(),
    "B": input("Enter the state of Room B (dirty/clean): ").lower()
}

```

```

def is_dirty(room):
    return rooms[room] == "dirty"

```

```

def clean_room(room):
    rooms[room] = "clean"

```

```

class VacuumAgent:

```

```

    def __init__(self, location):

```

```

        self.location = location

def move(self):
    if self.location == "A":
        self.location = "B"
    else:
        self.location = "A"

def perceive_and_act(self):

    if is_dirty(self.location):
        clean_room(self.location)
        return "Cleaned Room " + self.location

    else:
        self.move()
        return "Moved to Room " + self.location

print("\nInitial Environment:")
print("Room A:", rooms["A"])
print("Room B:", rooms["B"])

agent = VacuumAgent("A")

steps = int(input("\nEnter number of simulation steps: "))

print("\nSimulation Started")
print("-----")

for step in range(1, steps + 1):

    action = agent.perceive_and_act()

    print("\nStep", step)
    print("Agent Location:", agent.location)
    print("Action:", action)
    print("Room A:", rooms["A"])
    print("Room B:", rooms["B"])

print("\nSimulation Finished")
print("-----")
print("Final Environment:")
print("Room A:", rooms["A"])
print("Room B:", rooms["B"])

```

```
print("Agent Location:", agent.location)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. The agent follows rules based on the condition of its current room. If the room is dirty, the agent cleans it. If the room is clean, the agent moves to another dirty room. If all rooms are clean, the agent stops cleaning and reports that all rooms are clean.
2. The agent checks the condition of its current room. If the room is dirty, it cleans the room. If the room is already clean, it searches for other dirty rooms and randomly chooses one to move to.
3. If both rooms are already clean, the agent does not need to clean anything. It checks the environment and finds that there are no dirty rooms. The agent then reports that all rooms are clean.
4. I would store the rooms in a dictionary or list containing all the rooms and their states. The agent can check all the rooms for dirty conditions and randomly choose one of the dirty rooms to clean. This approach can be extended to three or more rooms without changing the basic rule-based behavior.